POLAND — Q3 COHORT

Final Project Overview

Choose one business challenge · design · build · present

Graduate Software Engineering Programme 2026 · Teams of 2–3

> **How to use this document.** This is the shared overview for all teams. Your chosen project also has a detailed **Project Brief** (HTML/PDF) with sprint deliverables, data model guidance, and starter `Data/` packs. Treat the brief as the baseline — additional asks may appear during the programme; Sprint 4 is for folding those in.

1. Overview

Your team designs and builds a financial services application based on **one** of the three project specifications below. You own the solution end to end: understand the business problem, model the data, expose an API, ship a usable frontend, and present the result.

Delivery runs as **four sprints**: Foundations (database + thin proof-of-concept API) → full API → finished full-stack app → final polish, innovation, and presentation.

2. The three projects (business view)

Pick one. Each is a realistic bank domain problem — different users, different pressures.

01	**Instrument Reference Data**	The bank's source of truth for financial instruments (equities, bonds, and related identifiers). Downstream systems rely on clean, consistent reference data. Errors multiply into wrong risk, reporting, and settlement. Focus: accurate master data, identifiers, and change history.
02	**KYC Client Onboarding**	Know Your Customer — verify identity and assess financial-crime risk before a client is onboarded. Compliance officers need cases, documents, and risk decisions with a clear audit trail. Focus: onboarding lifecycle that is thorough but workable under volume.
03	**Portfolio Risk & Exposure**	Monitor market risk across portfolios: exposure, Value-at-Risk (VaR), concentration, and limit breaches. Risk managers need to see problems early and act. Focus: positions, risk metrics, and breach handling.

3. What you are aiming for

- **Working product** — not a slide deck. Database, backend, API, and a simple frontend that hang together.
- **REST API** — create and retrieve the core records of your domain (and the other operations your brief requires).
- **Frontend** — enough UI for a real user of *your* domain to browse records, see useful status/metrics, and perform at least the key actions described in your brief. A clear, easy-to-use UI earns extra marks.
- **Innovation** — at least one thoughtful extra relevant to your domain (your brief lists optional prompts).

Use technologies from training (Java, SQL, JavaScript/TypeScript). Frameworks such as Spring Boot, React, or Vue are welcome if they help — check with your instructor if you want something outside the

training stack.

Further enhancements may be introduced once the core path is healthy. Your instructor acts as the customer: arrange short check-ins as needed.

> **Starter data.** Each project folder includes a `Data/` pack for exploring the domain. It is **not** the solution schema — design your own model; you may adapt useful sample rows into it.

4. Notes

1. User management is optional if time is short — a single assumed user is fine at first.
2. Use the database approach from training for persistence; other solid choices are acceptable if agreed.
3. Use Git well: branches and pull requests where you can. One shared repo per team.
4. Document how to run the app and call the API (OpenAPI/Swagger is a plus if you know it).
5. Strongly suggested: automated tests, CI, and Docker where they reduce risk — not for their own sake.

5. Getting started

Technical checklist

1. Create the project structure and a single GitHub repository for the team.
2. Invite instructors as viewers (username: `tuistmessiah`).
3. Set up Trello, Jira, or similar for tasks.
4. Commit and push a skeleton so everyone can clone and run the same baseline.
5. Agree the absolute minimum fields for a first working record — then grow. Stay agile; over-complex data models are the most common early failure.

Team checklist

1. Decide how you will split work (e.g. backend-first vs parallel UI/design).
2. Build a task list; prioritise a **minimal** vertical slice before polish.
3. Keep questions ready for instructor drop-ins.
4. Name the team, plan check-ins, protect energy — quality over quantity.

If you are blocked on any of the above, contact your instructor early.

6. Presentation

At the end of the programme you present to instructors and potentially managers / other stakeholders. **15 minutes** + **5 minutes** Q&A — same for every team size. Everyone speaks; cameras on. You are also expected to ask other teams questions.

Suggested story arc (adapt as you like):

- Introduce the team and the business problem you chose
- How you approached the work (roles, pairing, tools)
- Data model and high-level architecture (simple diagram is enough)
- Live demo of the main flows
- Challenges, what you would do differently, and what's next
- Thank you — questions

7. Teamwork

Teams are self-organising. Raise blockers with instructors promptly. Keep all work in one repository (public, or shared with every teammate and your instructors). Communicate regularly and review each other's work.

Start small. Ship something real. Build something you're proud of.